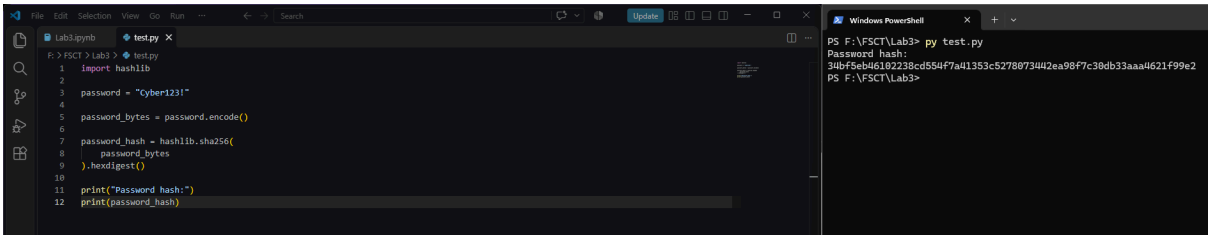
Part 2.1



```
import hashlib

password = "Cyber123!"

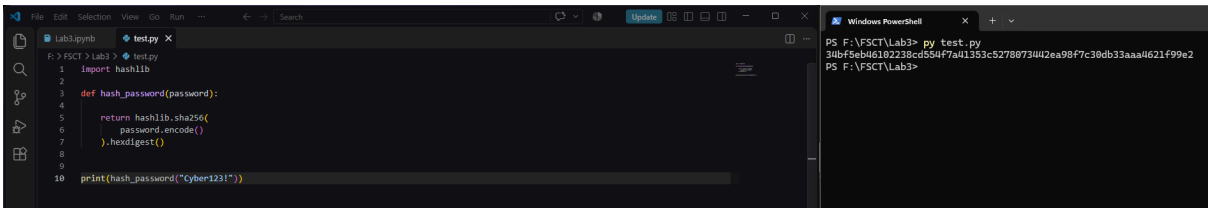
password_bytes = password.encode()

password_hash = hashlib.sha256(
    password_bytes
).hexdigest()

print("Password hash:")
print(password_hash)
```

```
PS F:\FSCT\Lab3> py test.py
Password hash:
34bf5eb46102238cd554f7a41353c5278073442ea98f7c30db33aaa4621f99e2
PS F:\FSCT\Lab3>
```

Part 2.2



```
import hashlib

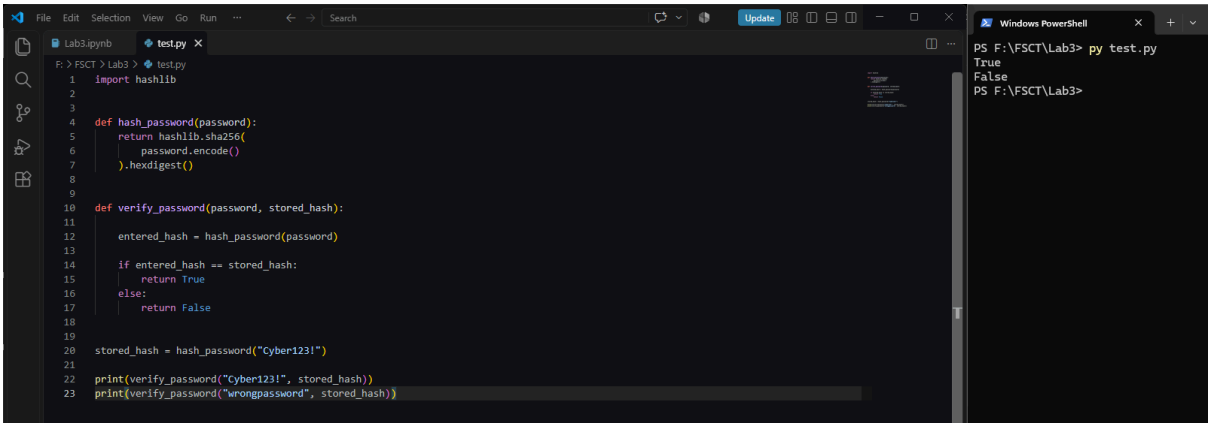
def hash_password(password):

    return hashlib.sha256(
        password.encode()
    ).hexdigest()


print(hash_password("Cyber123!"))
```

```
PS F:\FSCT\Lab3> py test.py
34bf5eb46102238cd554f7a41353c5278073442ea98f7c30db33aaa4621f99e2
PS F:\FSCT\Lab3>
```

Part 2.3



```
import hashlib


def hash_password(password):
    return hashlib.sha256(
        password.encode()
    ).hexdigest()


def verify_password(password, stored_hash):

    entered_hash = hash_password(password)

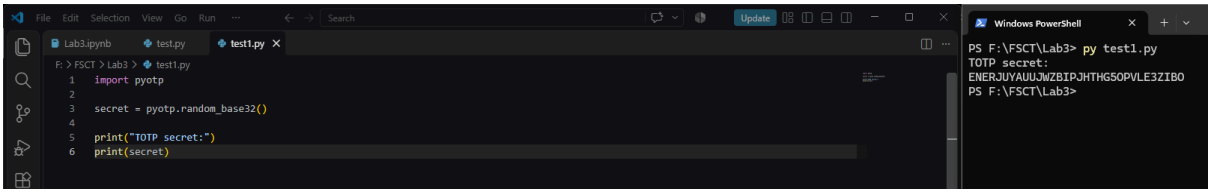
    if entered_hash == stored_hash:
        return True
    else:
        return False


stored_hash = hash_password("Cyber123!")

print(verify_password("Cyber123!", stored_hash))
print(verify_password("wrongpassword", stored_hash))
```

```
PS F:\FSCT\Lab3> py test.py
True
False
PS F:\FSCT\Lab3>
```

Part 5



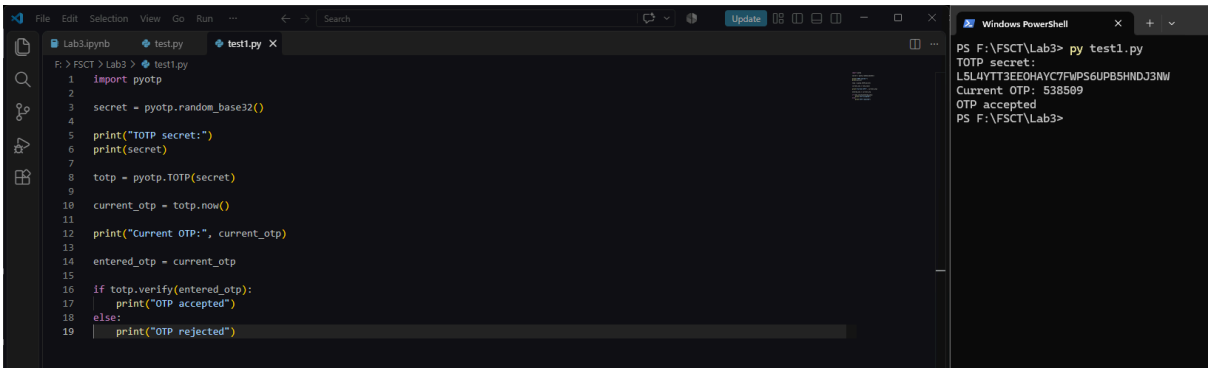
```
import pyotp

secret = pyotp.random_base32()

print("TOTP secret:")
print(secret)
```

```
PS F:\FSCT\Lab3> py test1.py
TOTP secret:
ENERJUYAUUJWZBIPJHTHG5OPVLE3ZIBO
PS F:\FSCT\Lab3>
```

Part 6



```
import pyotp

secret = pyotp.random_base32()

print("TOTP secret:")
print(secret)

totp = pyotp.TOTP(secret)

current_otp = totp.now()

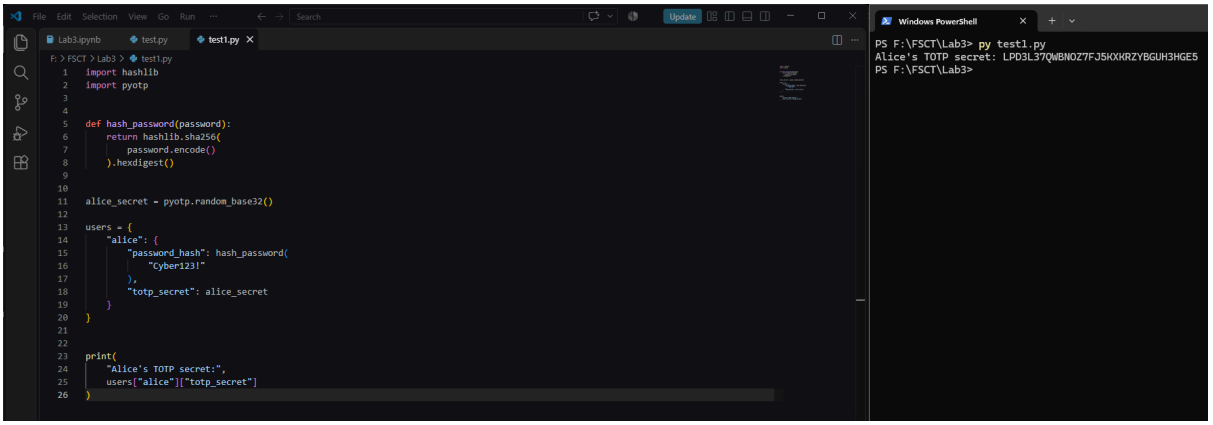
print("Current OTP:", current_otp)

entered_otp = current_otp

if totp.verify(entered_otp):
    print("OTP accepted")
else:
    print("OTP rejected")
```

```
PS F:\FSCT\Lab3> py test1.py
TOTP secret:
L5L4YTT3EEOHAYC7FWPSGUPB5HNDJ3NW
Current OTP: 538509
OTP accepted
PS F:\FSCT\Lab3>
```

Part 7



```
import hashlib
import pyotp


def hash_password(password):
    return hashlib.sha256(
        password.encode()
    ).hexdigest()

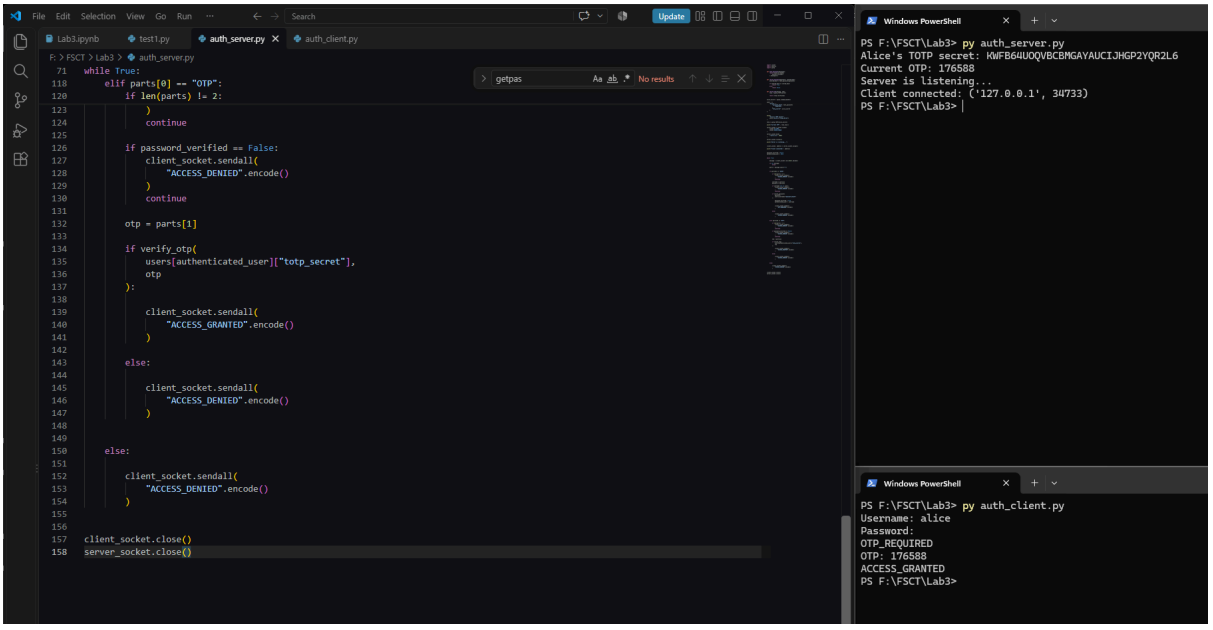

alice_secret = pyotp.random_base32()

users = {
    "alice": {
        "password_hash": hash_password(
            "Cyber123!"
        ),
        "totp_secret": alice_secret
    }
}


print(
    "Alice's TOTP secret:",
    users["alice"]["totp_secret"]
)
```

```
PS F:\FSCT\Lab3> py test1.py
Alice's TOTP secret: LPD3L37QWBNOZ7FJ5KXKRZYBGUH3HGE5
PS F:\FSCT\Lab3>
```

Part 12



```
while True:
    elif parts[0] == "OTP":
        if len(parts) != 2:
            )
            continue

        if password_verified == False:
            client_socket.sendall(
                "ACCESS_DENIED".encode()
            )
            continue

        otp = parts[1]

        if verify_otp(
            users[authenticated_user]["totp_secret"],
            otp
        ):

            client_socket.sendall(
                "ACCESS_GRANTED".encode()
            )

        else:

            client_socket.sendall(
                "ACCESS_DENIED".encode()
            )


    else:

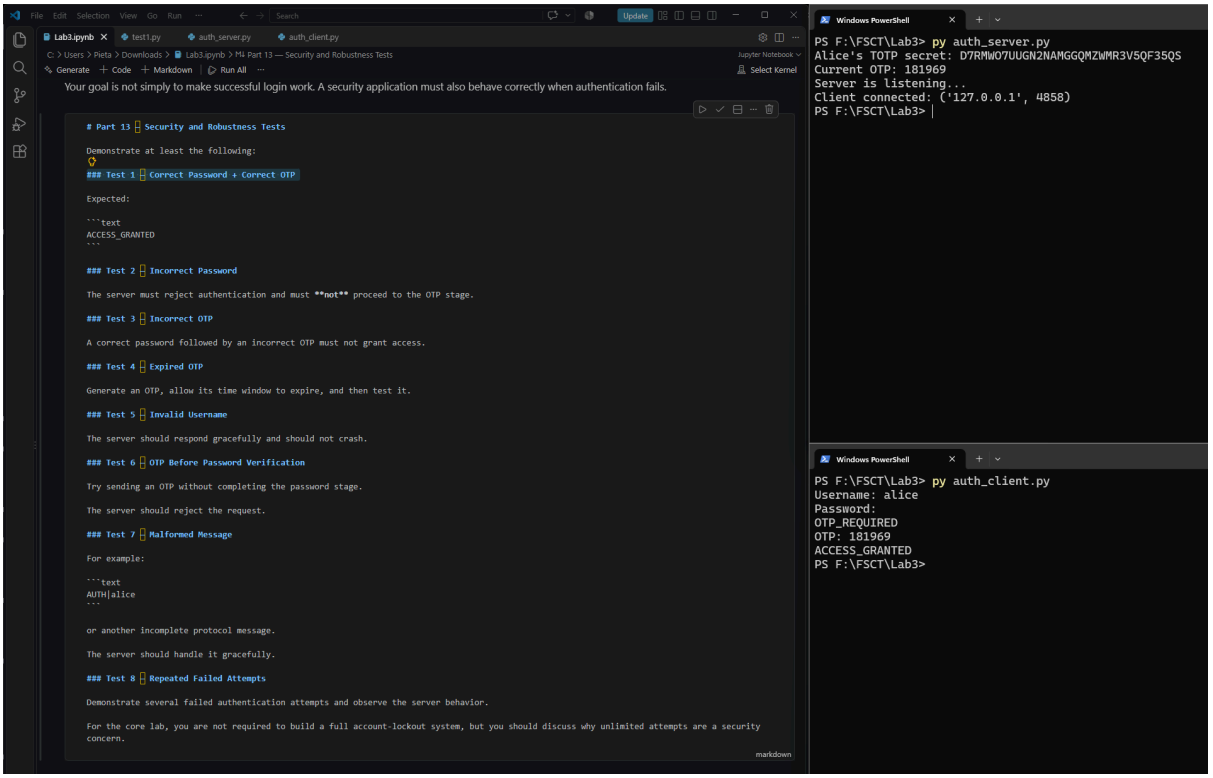
        client_socket.sendall(
            "ACCESS_DENIED".encode()
        )


client_socket.close()
server_socket.close()
```

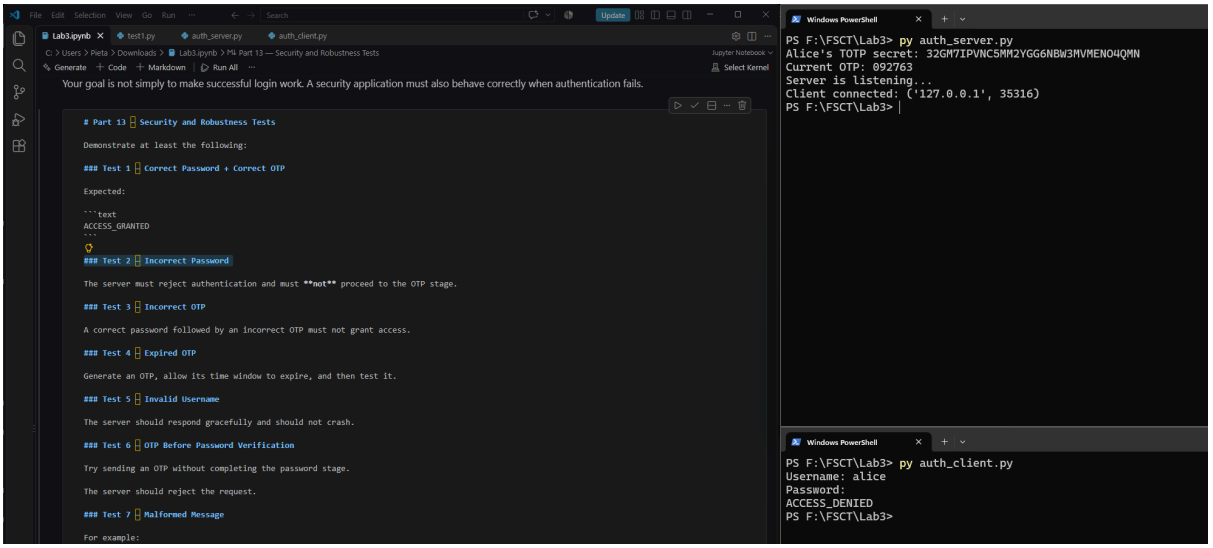
```
PS F:\FSCT\Lab3> py auth_server.py
Alice's TOTP secret: KWFB64UOQVBCBMGAYAUCIJHGP2YQR2L6
Current OTP: 176588
Server is listening...
Client connected: ('127.0.0.1', 34733)
PS F:\FSCT\Lab3>
```

```
PS F:\FSCT\Lab3> py auth_client.py
Username: alice
Password:
OTP_REQUIRED
OTP: 176588
ACCESS_GRANTED
PS F:\FSCT\Lab3>
```

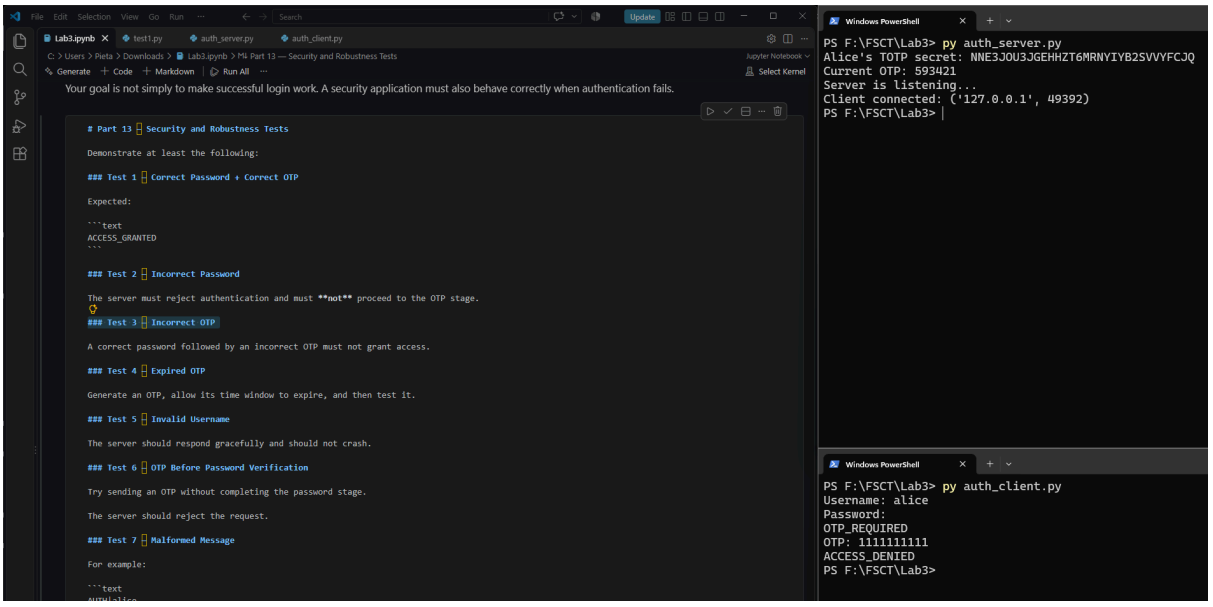
Part 13 - Test 1: Correct Password + Correct OTP



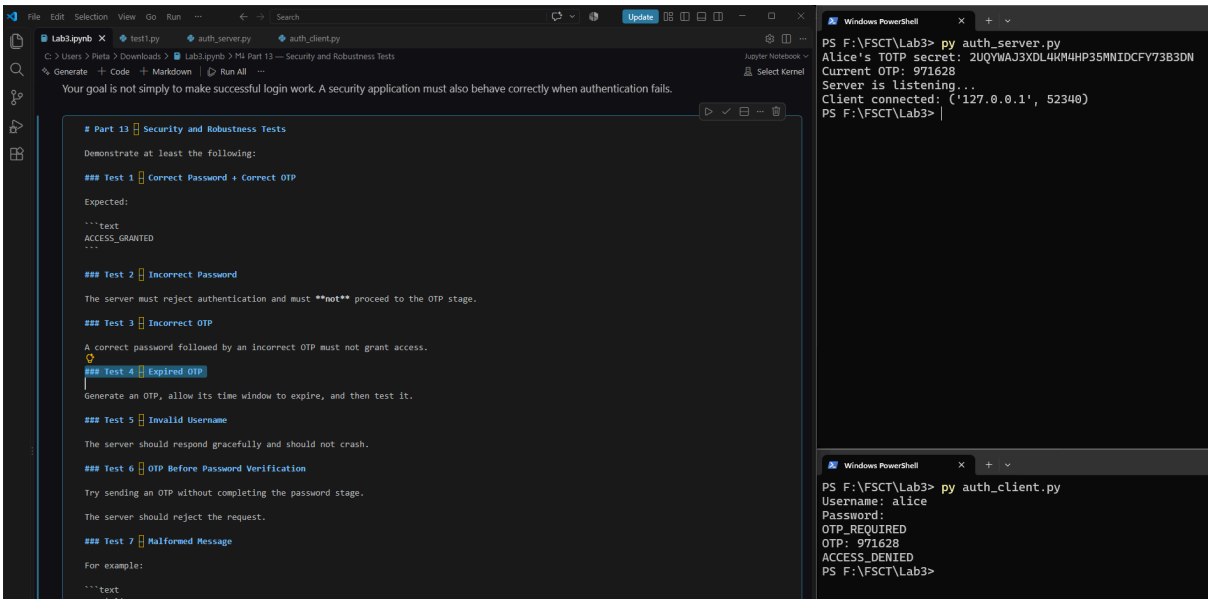
Part 13 - Test 2: Incorrect Password



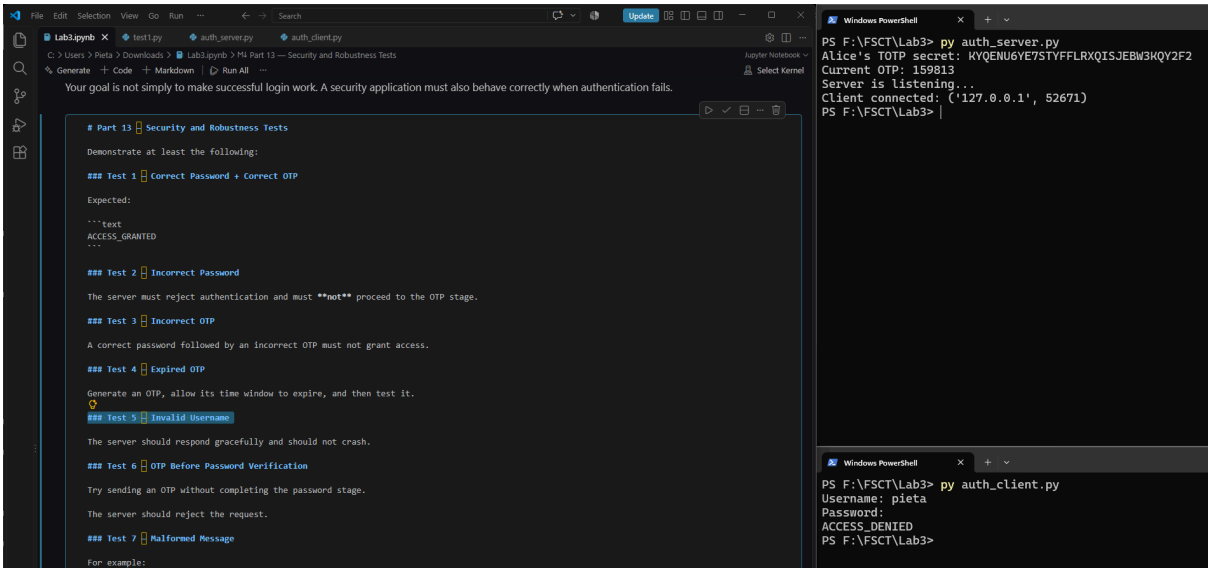
Part 13 - Test 3: Incorrect OTP



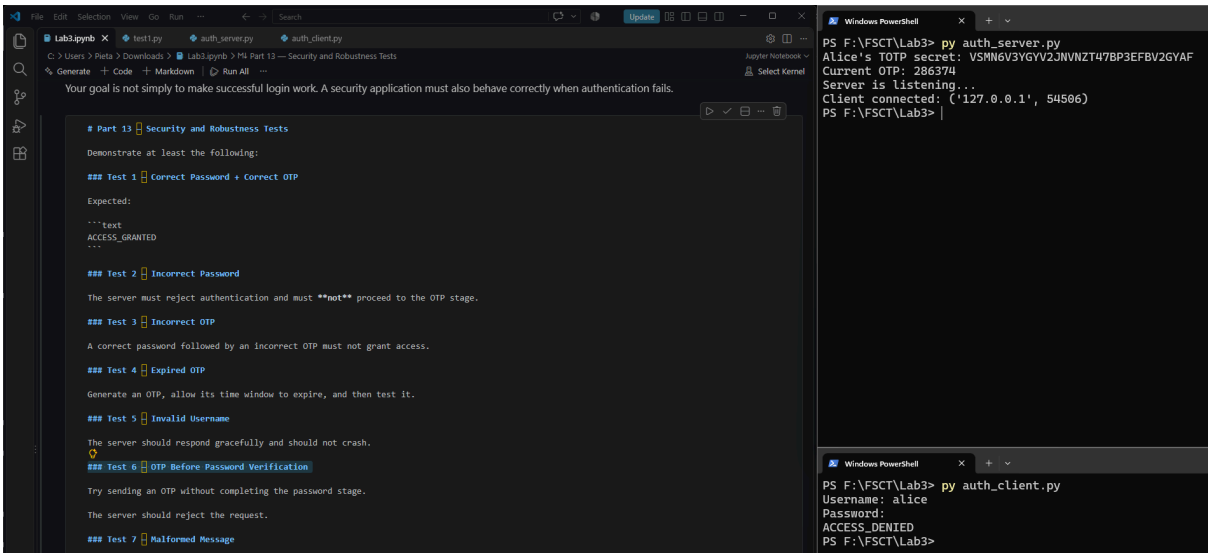
Part 13 - Test 4: Expired OTP



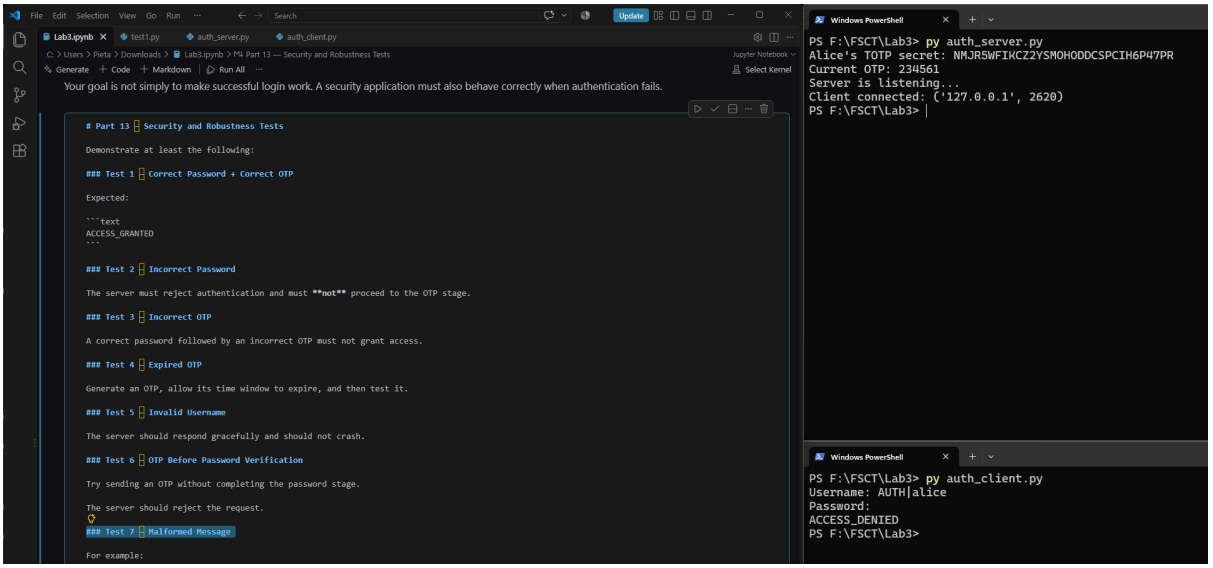
Part 13 - Test 5:



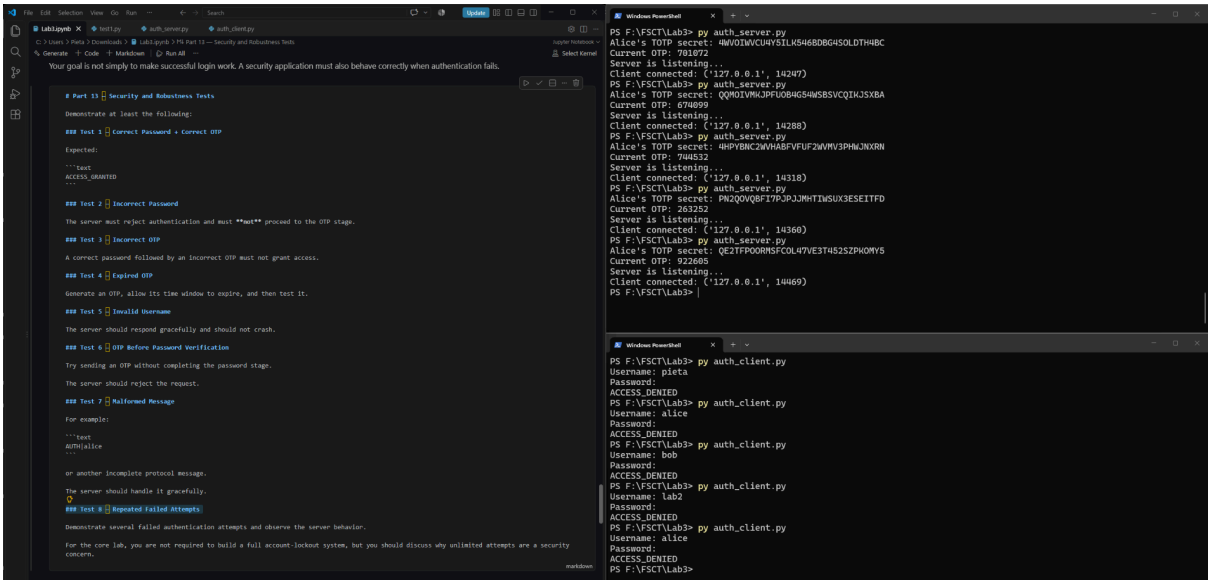
Part 13 - Test 6: OTP before password



Part 13 - Test 7: Malformed Message



Part 13 - Test 8: Repeated Failed Attempts



Part 15 - Security Reflection

1. Why should passwords not be stored in plaintext?

If the database is compromised, an attacker could immediately see and use the passwords.

2. What is the purpose of hashing a password before storage?

Hashing prevents the actual password from being stored directly. The hash can be compared during login instead.

3. Why is plain SHA-256 alone insufficient for production password storage?

SHA-256 is very fast, which makes it easier for attackers to guess many passwords quickly. A dedicated password-hashing algorithm should be used instead.

4. What additional protection does TOTP provide beyond a password?

TOTP provides a second authentication factor. An attacker would need both the password and the current OTP to log in.

5. How can TOTP reduce the usefulness of a captured one-time code after it expires?
The code is only valid for a short period of time. Once it expires, the captured code can no longer be used.
6. What happens when the user and server clocks are significantly out of sync?
The OTP may be rejected because TOTP depends on both devices having approximately the same time.
7. What happens if an attacker obtains the user's TOTP secret?
The attacker could use the secret to generate valid OTP codes.
8. Why should repeated failed login attempts be limited or monitored?
It helps prevent attackers from repeatedly guessing passwords or OTP codes.
9. Does `getpass()` protect the password while it travels across the network? Explain.
No. getpass only hides the password while it is being typed. It does not encrypt the password when it is sent over the network.
10. What additional security would be required before deploying this authentication system in the real world?
It would need encrypted network communication, stronger password hashing, secure storage of TOTP secrets, and protection against repeated login attempts.

Part 16
The authentication system in this lab uses a password and a TOTP code to make logging in more secure. Instead of saving the actual password, the server stores a SHA-256 hash of it. When someone logs in, the password they enter is hashed and compared to the saved hash. This is better than storing the password normally because someone who gets access to the stored data would not immediately see the actual password.

The TOTP adds a second step to the login. After the user enters the correct username and password, they also have to enter a six-digit OTP. The code changes about every 30 seconds, so an old code will not work for very long. This makes the system safer because even if someone knows the password, they would still need the current OTP to get access. There are still some security problems with my system. The client and server communicate using a normal TCP connection, which does not encrypt the information being sent. This means someone who intercepts the connection could possibly see the username, password, or OTP. In a real system, I would use TLS so that the information is encrypted while being sent.

Another problem is using SHA-256 by itself for passwords. It is very fast, which means an attacker could try guessing lots of passwords quickly if they got the hashes. The TOTP secret also needs to be kept safe. If someone steals it, they could generate their own valid OTP codes. The system should also limit repeated failed login attempts so someone cannot keep guessing passwords or codes.

Overall, the system is more secure than using only a password, but it would need improvements before being used in the real world. I would add encrypted communication, better password hashing, secure storage for TOTP secrets, and limits on failed login attempts.